

PostgreSQL 10-Day Complete Tutorial

Day 1: Introduction & Setup

- What is PostgreSQL?
- Features: Open-source, ACID compliant, extensible.
- Installation on Linux/Windows/Mac.
- Starting PostgreSQL service.
- Connecting via `\psql` and GUI tools like pgAdmin.

Exercise: Install PostgreSQL and connect using `\psql`.

Day 2: Database & Data Types

- CREATE DATABASE, DROP DATABASE.
 - Schemas and namespaces.
 - Common data types: INTEGER, TEXT, VARCHAR, DATE, BOOLEAN, JSONB.
 - Example: ``CREATE TABLE students (id SERIAL, name TEXT, age INT, enrolled DATE);``
- Exercise: Create a sample database and table.

Day 3: Basic CRUD

- INSERT: ``INSERT INTO students (name, age) VALUES ('Alice', 20);``
- SELECT: ``SELECT * FROM students;``
- UPDATE: ``UPDATE students SET age = 21 WHERE name = 'Alice';``
- DELETE: ``DELETE FROM students WHERE id = 1;``

Exercise: Insert 5 students and update one of them.

Day 4: Filtering & Joins

- WHERE, ORDER BY, LIMIT.
- Comparison operators: =, >, <, LIKE, IN.
- Joins: INNER, LEFT, RIGHT, FULL.

Example: ``SELECT s.name, c.course_name FROM students s JOIN courses c ON s.id = c.student_id;``

Exercise: Query students older than 20 and join with courses.

Day 5: Constraints & Indexes

- Primary Key, Foreign Key, Unique, Check, Not Null.
- Example: ``CREATE TABLE courses (id SERIAL PRIMARY KEY, course_name TEXT UNIQUE);``
- Indexing: ``CREATE INDEX idx_age ON students(age);``

Exercise: Add constraints to ensure unique course names.

Day 6: Transactions & ACID

- BEGIN, COMMIT, ROLLBACK.
- Savepoints.
- ACID properties in PostgreSQL.

Example:

```
```sql
```

```
BEGIN;
```

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
```

```
COMMIT;
```

```
```
```

Exercise: Perform a transaction with rollback.

Day 7: Views, Functions & Triggers

- View: `\CREATE VIEW active_students AS SELECT * FROM students WHERE age > 18;`

- Function:

```
```sql
```

```
CREATE FUNCTION student_count() RETURNS INT AS $$
```

```
SELECT COUNT(*) FROM students;
```

```
$$ LANGUAGE SQL;
```

```
```
```

- Trigger: Executes a function automatically.

Exercise: Create a trigger to log inserted students.

Day 8: Advanced Queries

- CTE (WITH clause).
- Window Functions (RANK, ROW_NUMBER, LAG, LEAD).
- Subqueries.

Example:

```
```sql
```

```
WITH recent_students AS (
 SELECT * FROM students WHERE enrolled > CURRENT_DATE - INTERVAL '30 days'
)
SELECT * FROM recent_students;
```
```

Exercise: Use RANK() to list students by age.

Day 9: JSONB, Full-Text Search, Partitioning

- JSONB: ``SELECT data->>'name' FROM products WHERE data->>'category'='Electronics';``
- Full-text search:

````sql`

`SELECT * FROM articles WHERE to_tsvector('english', content) @@ to_tsquery('database');`  
`````

- Partitioning: ``CREATE TABLE sales (...) PARTITION BY RANGE (sale_date);``
- Exercise: Create a JSONB column and query nested fields.

Day 10: Performance Tuning & Best Practices

- Use `EXPLAIN ANALYZE` for query optimization.
- Proper indexing and query rewriting.
- Normalization vs. denormalization.
- Connection pooling with PgBouncer.
- `VACUUM` and `autovacuum`.

Exercise: Optimize a slow query using `EXPLAIN ANALYZE`.